

Context compaction for small language models is a content-shape adapter: a falsification study

Ahmet Barış Günaydın *Independent researcher · github.com/abgnydn/butterfly*

Keywords: context compaction; context-window management; small language models; long-term memory; LongMemEval; pre-registration; falsification; reproducibility; LLM agents

Abstract

A recurring proposal for managing a small language model’s limited context window is **tag-and-rebuild compaction**: classify each message of a conversation as keep / summarize / melt, rebuild the survivors into a tight token budget (the “chrysalis”), inject fresh noise, and repeat across generations. The advertised claim is that this preserves load-bearing facts better than naive recency truncation (lastN) under compounding noise. We pre-registered that claim, **refuted our own first pre-registration**, filed a harder regime that **confirmed**, and then spent the remainder of the study attempting to break the confirmation. The surviving result is narrower and more useful than the original: tag-and-rebuild compaction is a **content-shape adapter, not a universal context manager — the engineering work lives in the tagger, not the rebuild step**. A regex tagger and two classifiers trained on its labels (45 and 2,307 parameters) reproduce the win exactly; frontier instruction-tuned LLM taggers (qwen3-14b, gemma) prompted to “find important messages” do not. On adversarial transcripts whose needles are real prose rather than identifier shapes, every tagger fails, and the multi-generation noise injection acts as a feedback loop that locks the protocol onto noise once the needle is dropped in the first cocoon. Validated against the external **LongMemEval** benchmark [1], the directional finding holds across three orders of magnitude of context size (6.6K → 1.25M tokens): a 14-parameter classifier trained on representative labels preserves 86 % of evidence turns at $3\times$ compression on the oracle split and beats lastN by +16 to +64 percentage points everywhere — yet the same classifier is **domain-locked** (0 % on engineering chat, whose needles it learned to ignore), and a downstream LLM-judged QA test shows a real $2.5\times$ lift over lastN on modest haystacks (38 % vs 15 %) that **collapses to 0–5 % on large multi-session haystacks** where the bottleneck is retrieval, not compaction. We report both refutations, the confirmation, and the boundary, and are explicit that the chrysalis loop — the original novel claim — was never validated downstream.

1. Introduction

Small, local language models exhaust their context window quickly, and the obvious remedy — drop the oldest tokens (lastN truncation) — discards load-bearing facts whenever they happen to be old. A more selective alternative *tags* each message by importance and *rebuilds* a compacted context from the survivors, iterating as new content arrives. It is intuitively appealing and easy to demonstrate: keep the file paths and decisions, melt the small talk, watch the important content survive.

Intuition and demonstrations are not evidence. This study treats the compaction mechanism as a falsifiable claim and runs it through a pre-registration discipline: each prediction is filed with a numeric confirm/refute threshold *before* the experiment, and the outcome is recorded regardless of direction. The pre-registrations carry a git-timestamped audit trail in the predictions log of the [neuropulse](#) visualizer, inside whose browser demo the mechanism first ran (entries P-20260512-05, P-20260515-06).

Contributions. This study contributes:

1. **A pre-registered falsification** of the broad tag-and-rebuild claim (refuted twice) and confirmation of a narrow regime, with the phase boundary between them mapped (§3.1–3.2).

2. **A localization result:** across regex, two trained classifiers, and frontier LLM taggers, the win lives in the *tagger’s prior*, not the rebuild step — demonstrated by exact reproduction under learned taggers and total failure under a frontier LLM and on prose-needle adversarial transcripts (§3.3–3.4).
3. **External, cross-scale validation** against LongMemEval [1] over three orders of magnitude of context, including the domain-lock and downstream-QA boundaries that fence the claim in (§3.5–3.6).
4. **A negative result stated honestly:** the multi-generation chrysalis loop — the mechanism’s original selling point — was never validated downstream, and the technique is not a long-term memory system (§4).

The contribution is not a new memory architecture; it is a map of *when* tag-and-rebuild compaction beats truncation and *why*, obtained by refuting the broad claim before defending the narrow one.

2. Methods

2.1 Mechanism and notation

A conversation is an ordered list of messages $C = (m_1, \dots, m_n)$. A **tagger** is a function $\tau : m_i \mapsto \{\text{keep, summarize, melt}\}$. The **chrysalis** rebuild at token budget B is

$$R_B(C, \tau) = \text{trunc}_B \left(\bigoplus_{i: \tau(m_i)=\text{keep}} m_i \oplus \bigoplus_{i: \tau(m_i)=\text{summarize}} s_1(m_i) \right),$$

where $\oplus$ is order-preserving concatenation, s_1 takes the first sentence, melted messages are dropped, and trunc_B caps the result at B tokens. One **generation** is a tag-then-rebuild pass; between generations, off-topic noise messages are injected, and the original demo ran $G = 3$. Writing $p(\cdot)$ for the fraction of a transcript’s load-bearing needle keywords present in a compacted context, the quantity of interest is

$$\Delta = p(R_B(C, \tau)) - p(\text{lastN}_B(C)),$$

the needle-preservation advantage of butterfly over recency truncation at the same budget. lastN_B keeps the most recent messages up to B tokens.

2.2 Taggers

Five taggers are evaluated under matched protocol: (i) a hand-tuned **regex** tagger that fires **keep** on identifier shapes (file paths, ticket IDs, channel names, package mentions, line ranges, decision markers) and **melt** on bare acknowledgements and short tangents; (ii) a **14-feature softmax classifier** (45 parameters) trained on the regex’s labels; (iii) a **768-dimensional embedding + linear head** (2,307 parameters, nomic-embed features) trained the same way; (iv) **frontier instruction-tuned LLM taggers** (qwen3-14b, gemma-4) prompted to select important messages, across several prompt and output-format strategies; and (v) a **14-feature classifier trained on the LongMemEval oracle’s own has_answer labels** (~11K labeled turns).

2.3 Datasets, scoring, and protocol

The pure-code experiments score **needle-keyword coverage** — substring match of the load-bearing identifiers in each transcript’s expected fact against the compacted context — which is deterministic and needs no LLM judge. Four hand-written engineering-chat transcripts carry identifier-shaped needles; four **adversarial** transcripts (a numeric threshold, a relative deadline, a stated preference, a buried causation) carry prose needles by construction. Regimes are specified as (length, budget, generations); the *easy* regime is (12, 400, 1) and the *hard* regime is (38, 100, 3). External validation uses **LongMemEval** [1] (500 conversations; evidence turns flagged has_answer), whose three splits span the scale axis: **oracle** (~6.6K tokens, evidence local), **s** (~121K tokens, 2 evidence turns in 550), and **m** (~1.25M tokens, 1 evidence turn

in 5,057); the metric is the fraction of evidence turns surviving compaction (*turn-rate*), with a secondary *answer-string-in-memory* metric. The downstream test feeds the compacted memory plus the question to an answerer (qwen3-4b) and grades the answer with two independent LLM judges (qwen3-4b and gemma-4) to control for same-model bias. All pure-code runs are deterministic; LLM-tagger and QA runs use a local LM Studio endpoint on an Apple M2 Pro.

3. Results

3.1 Two pre-registrations: refuted, then confirmed

P-20260512-05 (filed): at the original demo regime (12, 400, 1), needle preservation should beat lastN by ≥ 15 pp across four transcripts. **Refuted.** lastN tied butterfly on two transcripts and beat it on the other two. At a generous budget on short transcripts, recency truncation already captures the needle and the mechanism has nothing to differentiate. (The original Phi-3-in-browser harness could not gather the pre-registered sample under background-tab throttling — six attempts, one completed run — so the protocol was re-run in pure code with the regex tagger, 4 ms total, identical transcripts and scoring.)

P-20260515-06 (filed): file a regime where compaction must matter — the hard regime (38, 100, 3) with fresh noise each generation. **Confirmed.** All four transcripts: butterfly preserves 100 % of needle keywords, lastN 0 %, mean $\Delta = 100$ pp.

3.2 The phase boundary

Two binary outcomes do not say *where* the mechanism stops mattering. A sweep of 8 budgets $\times$ 6 lengths $\times$ 5 generation counts $\times$ 4 transcripts (960 cells, 73 ms) locates a roughly diagonal boundary (Figure 1): butterfly stops winning once the budget exceeds $\sim 30\text{--}40\%$ of the original transcript size, because lastN then already fits the needle; more generations expand the win region. The refuted point of §3.1 sits in the tie zone and the confirmed point deep in the win zone — both are samples from a real phase structure, not luck.

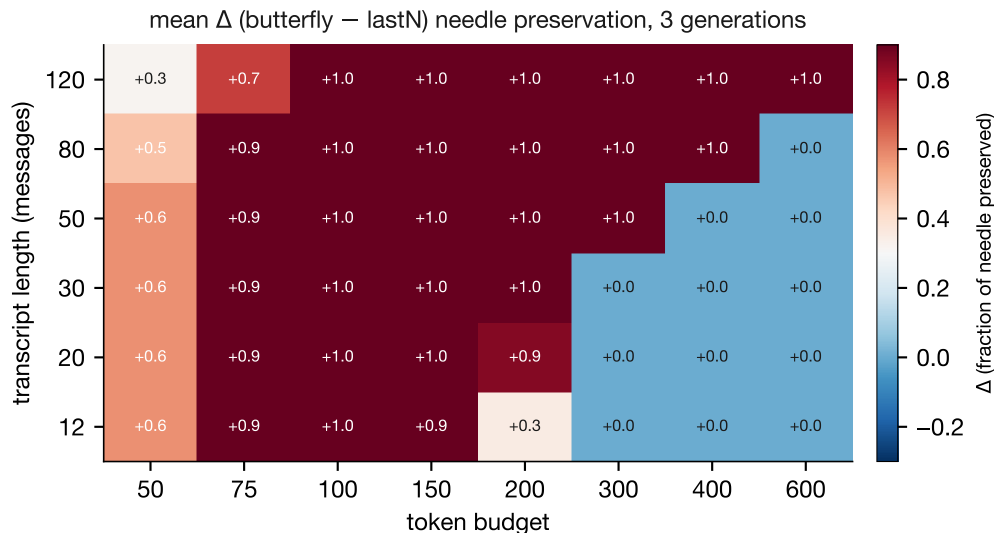


Figure 1: Where the mechanism matters. Mean Δ (butterfly — lastN needle preservation, averaged over four transcripts) at three generations, across transcript length and token budget. Red cells are butterfly wins; the win region is the tight-budget / long-transcript corner and shrinks as the budget grows past $\sim 30\text{--}40\%$ of transcript size.

3.3 Swapping the tagger: the win lives in the prior

The natural objection is that the regex is hand-fit to four transcripts. It is not the source of the win: two classifiers trained on the regex’s labels reproduce it exactly, while a frontier LLM tagger does not (Table 1).

Table 1: Hard-regime (38, 100, 3) result by tagger. Learned classifiers recover the regex boundary by gradient descent; the frontier LLM does not.

Tagger	Parameters	Hard-regime result
regex (hand-tuned)	~14 rules	100/0 , $\Delta = 100$ pp
14-feature softmax	45	100/0 , $\Delta = 100$ pp
768-dim embed + linear	2,307	100/0 , $\Delta = 100$ pp
qwen3-14b, one-char, cap=3	frontier	0/0, $\Delta = 0$ pp

The LLM over-tags (52–63 % of messages marked keep versus the regex’s ~8 %), bloating the chrysalis past budget, and on JSON-format failures falls back to a regex that cannot read its own rebuilt string. Four further LLM configurations (output shape, keep-caps, a smaller model, identifier-first prompts) all failed: with a hard cap the LLM picks three messages, just not the needle-carrying ones — it prioritizes decision-language, while the needles are literal identifier shapes. Both learned classifiers, by contrast, recover the regex’s boundary; this is partly tautological (they train on regex labels) but informative — the win is in the *feature distribution the tagger weighs*, not in pathological rule code.

3.4 Adversarial transcripts: every tagger fails, and noise compounds

If the win is shape-specific, transcripts whose needles are real load-bearing prose that does *not* take identifier shapes should break every tagger. On the four adversarial transcripts at the hard regime, every tagger’s mean Δ is 0 pp: the regex and its learned descendants tag every message `me!t` (no feature fires), and the LLM picks five messages, none the needle. A second finding the confirmation did not show: across all four adversarial transcripts the LLM tagger’s generation-3 output is *identical* — a noise message injected at generation 2. The multi-generation loop is a feedback amplifier; once the needle is dropped in the first cocoon, no later generation recovers it, and the only content that survives is whatever the tagger consistently labels keep. If that is noise, the protocol locks onto noise.

3.5 External validation: LongMemEval across three orders of magnitude

On the oracle split (500 real conversations), the regex tagger wins on independent data — turn-rate 21.4–29.0 % across budgets 256–2048 versus `lastN`’s 3.3–23.4 %. Two effects appear at once: classifiers trained on the tiny in-domain set *overfit* (the 45-parameter flat at ~4.7 %, the 2,307-parameter at 0 %), and different metrics have different winners — `lastN` wins answer-string-in-memory at high budgets (116/479 vs 63/479 at budget 2048), because answers often live in recent turns. A 60/40 **hybrid** of chrysalis and a `lastN` window matches or beats regex on turn-rate at budgets ≥ 512 and lifts answer-recall to 99/479.

Training the 14-parameter classifier on LongMemEval’s *own* `has_answer` labels reverses the overfitting and scales (Figure 2, Table 2): same 1.2 KB classifier file, three peer-reviewed splits, consistent ordering (longmem-trained > longmem-hybrid > regex > `lastN`) at every budget. On `m` — one evidence turn in 5,057 — it preserves 19 % of evidence at 2.5 % of the original context, $7\times$ `lastN`’s rate.

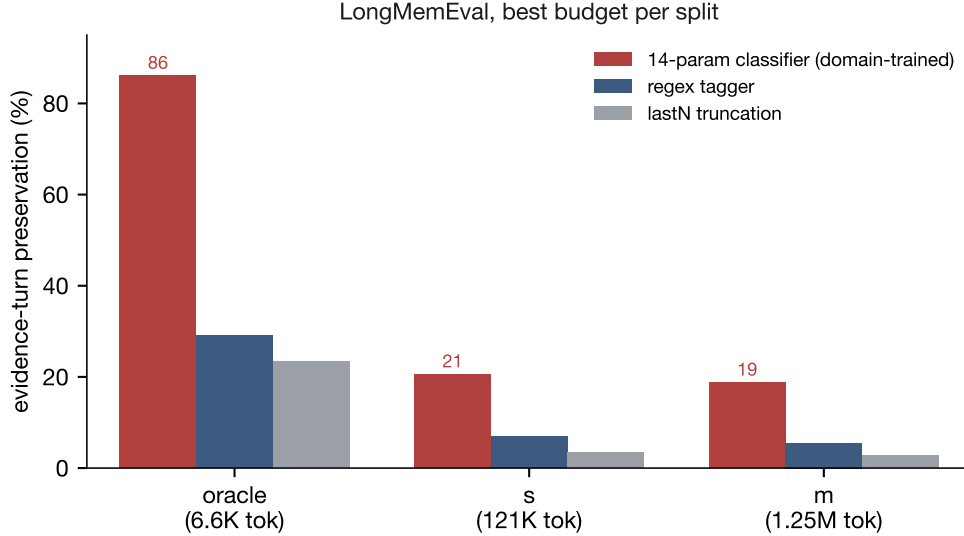


Figure 2: Cross-scale evidence preservation on LongMemEval. A 14-parameter classifier trained on representative labels beats both the regex tagger and lastN truncation at the best budget of every split, from 6.6K to 1.25M tokens of context; absolute rates fall as the needle-to-haystack ratio worsens, but the ordering is invariant.

Table 2: LongMemEval cross-scale summary. Best-budget turn-rate for the domain-trained classifier versus lastN, by split.

Split	Avg context	longmem-trained @ best budget	vs lastN
oracle	6.6K tok	86.1 % @ 2K ($3\times$ compression)	+63 pp
s	121K tok	20.6 % @ 4K ($30\times$ compression)	+17 pp
m	1.25M tok	18.8 % @ 32K ($39\times$ compression)	+16 pp

3.6 The boundaries: per-type loss, domain-lock, and downstream collapse

Three results fence the claim in. **Per-question-type** (oracle, budget 2048): the trained classifier wins 5 of 6 LongMemEval question types by 53–93 pp and *loses* single-session-assistant by 29 pp, because its dominant learned weight ($\log \text{length} \approx -3.25$) down-weights long messages — correct when evidence is short user statements, wrong when evidence is a long assistant response. **Domain-lock**: the LongMemEval-trained classifier has *negative* weights on identifier features (`file_path`, `decision_kw`, `proper_name`), the opposite of what engineering chat needs, and preserves ~5 % of needles on the original engineering transcripts versus the regex’s 100 %; the 14-feature template encodes either prior, and the training data picks which. Production butterfly is therefore *one classifier per deployment domain*. **Downstream QA**: substring preservation overstates usefulness by $\sim 2\times$; with an LLM answerer and two judges on oracle, the trained tagger reaches 38 %/29 % answer accuracy versus lastN’s 15 %/7 % (inter-judge agreement 87–93 %), a real $2.5\times$ lift — which then **collapses to 0–5 % on the s split** (50-session, ~ 50 –100K-token haystacks), where tag-by-message compaction cannot surface the right session and retrieval is the true bottleneck.

4. Limitations and threats to validity

- **The chrysalis loop was never validated downstream.** The QA evaluation used single-pass tag-and-rebuild; whether iterated multi-generation compaction with noise beats single-pass at downstream

QA — butterfly’s *original* novel claim — remains untested. Every confirmation here is of the tagger, not the loop. This is the central construct-validity threat.

- **Not a long-term memory system.** The mechanism fails on large multi-session haystacks (LongMemEval s, m) where the bottleneck is retrieval, not compaction.
- **The result is tagger-prior-specific.** The classifier inherits whatever its training labels prioritize; cross-domain transfer is $\sim 0\%$. Per-domain labels or a multi-domain corpus are required to deploy.
- **lastN is a weak baseline.** Production memory systems use vector retrieval plus LLM summarization [2,3]; beating lastN is necessary, not sufficient, for production relevance, and those systems were not benchmarked — the main external-validity threat.
- **Pure-code scoring is substring-based.** Needle-keyword coverage is deterministic but overstates downstream usefulness by $\sim 2\times$ (§3.6); the LLM-judged QA results are the load-bearing ones.
- **Not a substitute for frontier /compact.** When a frontier model is available, its summarization is cheaper and better; the “small local classifier
– small local answerer” lane is for the local-first niche.

5. Related work

LongMemEval [1] and MemoryAgentBench [2] are the external references for long-term conversational memory; this study uses LongMemEval as an independent oracle rather than proposing a competing system. Production memory frameworks — MemGPT/Letta [3] and mem0 — combine vector retrieval with LLM summarization and are the real prior art for deployment; we benchmark only against fixed-budget recency truncation and are explicit (§4) that this is a weak baseline. Context compaction by importance tagging is folklore in agent tooling; the contribution here is to subject it to pre-registered falsification, to localize its power in the tagger’s prior rather than the rebuild step, and to map its failure boundaries (adversarial shapes, domain-lock, large-haystack retrieval) against an external benchmark across three orders of magnitude of context size. The small-model under test is Phi-3-mini [4].

6. Reproducibility and software availability

Source: github.com/abgnydn/butterfly (MIT). The hard-regime confirmation runs in pure code with no LLM and no GPU — `node butterfly-purecode-hard.mjs`, 4 ms, deterministic. The phase sweep (`butterfly-sweep-phasediagram.mjs`), the tagger comparison (`butterfly-llm-tagger.mjs`), the trained taggers (`butterfly-train-classifier.mjs`, `-embed.mjs`, `-on-longmem.mjs`), the adversarial transcripts (`butterfly-adversarial.mjs`), the LongMemEval harness (`butterfly-longmemeval.mjs`, `butterfly-crossdomain.mjs`), and the downstream-QA evaluator (`butterfly-qa-eval.mjs`) reproduce every number above; both figures are generated from the committed sweep artifact and the LongMemEval result tables. LLM-tagger and QA runs use a local LM Studio endpoint. The early-April origin harness (`experiment.mjs`, `transgen.mjs`, `hybrid.mjs`) is preserved in the same repository.

Data availability. Pre-registrations with their git-timestamped audit trail are in the neuropulse predictions log (PREDICTIONS.md, entries P-20260512-05, P-20260515-06). The LongMemEval data [1] is third-party and obtained from its authors’ release; it is not redistributed here.

Author contributions. A.B.G. is the sole author and conducted all design, implementation, experiments, and writing.

Generative-AI disclosure. Portions of the software, documentation, and this manuscript were drafted with a large language model used as a coding and writing aid; all output was author-reviewed, correctness was enforced by deterministic keyword-coverage scoring and the committed per-generation traces, and every quantitative claim is traceable to a recorded run or an external benchmark.

Statements. Sole author; no competing interests; no external funding.

7. Conclusion

Tag-and-rebuild context compaction for small language models is real but narrow: it beats recency truncation at tight budgets under compounding noise **only when the tagger’s prior matches the load-bearing-content distribution of the transcripts being compacted**. The engineering problem is the tagger, not the rebuild step — a 45-parameter classifier trained on a few hundred labeled examples of a domain’s load-bearing shapes outperforms both hand-coded rules and zero-shot frontier-LLM tagging, and the win holds against an external benchmark from 6.6K to 1.25M tokens of context. But the same classifier is domain-locked, the mechanism fails where retrieval rather than compaction is the bottleneck, and the multi-generation loop that was its original selling point was never validated downstream. The useful deliverable is a recipe with a stated domain of validity, arrived at by refuting the broad claim before defending the narrow one.

References

- [1] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu. “LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory.” *Int. Conf. on Learning Representations (ICLR)*, 2025. *arXiv:2410.10813*. <https://arxiv.org/abs/2410.10813>
 - [2] Y. Hu et al. “Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions” (MemoryAgent-Bench). *arXiv:2507.05257*, 2025. <https://arxiv.org/abs/2507.05257>
 - [3] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. “MemGPT: Towards LLMs as Operating Systems” (now Letta). *arXiv:2310.08560*, 2023. <https://arxiv.org/abs/2310.08560>
 - [4] M. Abdin et al. (Microsoft). “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone.” *arXiv:2404.14219*, 2024. <https://arxiv.org/abs/2404.14219>
 - [5] mem0: production memory layer for AI agents. <https://github.com/mem0ai/mem0>
-

Draft v0.2 — companion to the neuropulse visualizer (github.com/abgnydn/neuropulse), inside whose browser demo the mechanism first ran. Built from the committed harness and per-generation traces; LLM-tagger and downstream-QA numbers come from local LM Studio runs. To be converted to a venue LaTeX template before submission to an NLP/ML-systems or agents venue.